Date: _____

University Of Karachi (UOK)
Department of Computer Science (UBIT)

Course: Operating Systems

Assignment No: 03

Semester: 4th BSSE Evening (A)

Submitted By:

Name: Shaikh Mahad Ud Din

Seat No: EB25210106113

Submitted To:

Ms. Nazish Imran

Submission Date:

PAPER PRODUCT

Scanned with CamScanner

Date:_____

1. Process Synchronization

1.1) Definition :-

Process synchronization is the coordination of concurrent processes or threads so that they access shared data and resources in a controlled and correct manner and execute dependent operations in the required order.

Synchronization is required because concurrent processes or threads may access the same data at unpredictable times. Without proper coordination, this may produce incorrect or inconsistent results.

1.2) Need for Synchronization :-

Synchronization is used to :

- Prevent incorrect simultaneous modification of shared data.
- Prevent race conditions.
- Maintain data consistency.
- Control access to shared resources.
- Ensure dependent operations execute in the correct order.

1.3) Race Condition :-

A race condition is a situation in which two or more concurrent processes or threads access shared data, atleast one of them modifies it, and the final result depends on the unpredictable order or timing of their execution.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Example:

Initial: counter = 5

T_1 reads counter = 5
T_2 reads counter = 5

T_1 increments and writes 6
T_2 increments and writes 6

Expected result = 7
Actual result = 6

The error occurs because counter++ may internally involve separate read, modify, and write operations.

1.4) Critical Section :-

A critical section is the part of a process or thread's code in which shared data or shared resource is accessed or modified and therefore requires protection from conflicting concurrent access.

Entry Section
↓
Critical Section
↓
Exit Section
↓
Remainder Section

SP PAPER PRODUCT

Scanned with CamScanner

Date:

- Entry Section: requests permission to enter
- Critical Section: accesses the protected shared resource.
- Exit Section: releases permission
- Remainder Section: performs the remaining non-critical work.

1.5) Requirement of a Correct Critical-Section Solution :-

- **Mutual Exclusion**: If one process or thread is executing in its critical section, no other conflicting process or thread may execute its corresponding critical section at the same time.
- **Progress**: If the critical section is free and one or more processes wish to enter, the decision about who enters next must not be postponed indefinitely.
- **Bounded Waiting**: After a process requests entry into its critical section, there must be a finite limit on the number of times other processes may enter before its request is granted.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

2. Semaphores :-

A semaphore is a synchronization variable used to coordinate concurrent processes or threads and control access to shared resources through atomic operations called wait() and signal().

The two operations are :

wait(S) → acquire / request
signal(S) → release / notify

They are also traditionally called :

P → wait()
V → signal()

2.2) wait() Operation :-

The wait() operation attemps to acquire a resource or permission.

```
wait(S)
{
    while(S <= 0);
    S--;
}
```

If the required resource is unavailable, the process must wait.

PAPER PRODUCT

Scanned with CamScanner

Date:

2.3) signal() Operation :-

The signal() operation releases a resource or permission.

```
signal(S) { S++; }
```

In a blocking implementation, signal() may also wake a process waiting on the semaphore.

2.4) Types of Semaphores :-

Binary Semaphore :-

A binary semaphore has two logical states, usually 0 and 1, and is commonly used to provide mutual exclusion.

1 → resource available
0 → resource unavailable

Example :

```
semaphore mutex = 1;

wait(mutex);

/* Critical Section */

signal(mutex);
```

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Counting Semaphore :-

A counting semaphore represents the number of available instances of a resource and is used when several identical resources may be accessed concurrently.

Example: For 5 identical printers :

S = 5

Each acquired printer decreases S, while each released printer increases S.

2.5 Binary vs Counting Semaphore :-

Binary Semaphore	Counting Semaphore
• Values normally 0 or 1	Represents multiple resource instances.
• Mainly used for mutual exclusion.	Mainly used for resource management.
• Usually initialize to 1.	Usually initialized to number of resources.
• Example: critical-section lock.	Example: pool of printers

PAPER PRODUCT

Scanned with CamScanner

Date:

3. Producer - Consumer Problem :-

3.1 Definition :-

The Producer - Consumer Problem is a classical synchronization problem in which producer generates items and placed itthem into a shared buffer, while a consumer removes and uses those items. Access to the buffer must be synchronized to prevent incorrect concurrent operations.

In a bounded buffer, the number of storage slot is limited.

Therefore:

- Producer must wait when the buffer is full.
- Consumer must wait when the buffer is empty.
- Buffer modification must be mutually exclusive.

3.2 Semaphores Used :-

For a buffer containing N slots:

```
semaphore mutex = 1;    // Protect buffer
semaphore empty = N;    // count of empty slots
semaphore full = 0;     // count of filled slots
```

- mutex: Protects access to the shared buffer.
- empty: Counts the number of empty slots.
- full: Counts the number of occupied slots.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Producer Code :-

```
while(true)
{
 item = produce_item();

 wait(empty);
 wait(mutex);

 insert_item(item);

 signal(mutex);
 signal(full);
}
```

wait(empty) ensures that space exists, while wait(mutex) protects the actual buffer modification.

Consumer Code :-

```
while(true)
{
 wait(full);
 wait(mutex);

 item = remove_item();

 signal(mutex);
 signal(empty);

 consume_item();
}
```

wait(full) ensures that an item exists before the consumer attempts to remove one.

3.4 Important Point :-

The three semaphores perform different jobs:

mutex → Who may access the buffer?

empty → Is there space for a new item?

full → Is there an item available to consume?

PAPER PRODUCT

Scanned with CamScanner

Date:

4. Readers - Writers Problem :-

4.1 Definition :-

The Readers - Writers Problem is a classical synchronization problem in which multiple processes or threads share a common data object. Reader processes only read the data, while writer processes modify it. Multiple readers may read simultaneously, but a writer requires exclusive access.

Access Rules :

Reader + Reader ⟶ Allowed
Reader + Writer ⟶ Not Allowed
Writer + Writer ⟶ Not Allowed

* Therefore: Any number of readers may read concurrently, but writing requires exclusive access.

4.2 Variables Used :-

```
int read_count = 0;
semaphore mutex = 1;
semaphore rw_mutex = 1;
```

- read_count : Stores the number of active readers.
- mutex : Protects modifications to read_count.
- rw_mutex : Control access to the actual shared data and prevents writers from entering while readers are active.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

4.3 Writer Code :-

A writer must obtain rw_mutex, giving it exclusive access to the shared data.

```
while (true)
{
    wait(rw_mutex);
    /* Write shared data */
    signal(rw_mutex);
}
```

4.4 Reader Code :-

```
while (true) {

    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);

    signal(mutex);

    /* Read shared data */

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);

    signal(mutex);
}
```

SP PAPER PRODUCT

Scanned with CamScanner

Date:

4.5 First Reader / Last Reader Principle :-

The central idea of the solution is :

First Reader
↓
wait (rw_mutex)
↓
Writers are blocked

Additional readers may then read concurrently. When readers begin leaving :

Last Reader
↓
signal (rw_mutex)
↓
Writers may enter

Therefore : The first reader blocks writers out, while the last reader allows writers to enter again.

4.6 Writer Starvation :-

The first Readers - Writers solution gives preference to readers. If new readers continue arriving before all existing readers leave, a waiting writer may be delayed indefinitely. This is called writer starvation.

PAPER PRODUCT

Scanned with CamScanner